EXP 1 Substitution and transposition cipher

Server.java

```
import java.io.*;
import java.net.*;

public class Server {
public static void main(String[] args) throws Exception {

ServerSocket ss = new ServerSocket(5000);
System.out.println("Server started. Waiting for client...");

Socket s = ss.accept();
System.out.println("Client connected.");

BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
PrintWriter out = new PrintWriter(s.getOutputStream(), true);

String choice = in.readLine();
String text = in.readLine();
int key = Integer.parseInt(in.readLine());

String encrypted = "";
String decrypted = "";

if (choice.equals("1")) {
encrypted = caesarEncrypt(text, key);
decrypted = caesarDecrypt(encrypted, key);
}
else if (choice.equals("2")) {
encrypted = transpositionEncrypt(text, key);
decrypted = transpositionDecrypt(encrypted, key);
}
else {
out.println("Invalid choice");
s.close();
ss.close();
return;
}

out.println("Original Text: " + text);
out.println("Encrypted Text: " + encrypted);
out.println("Decrypted Text: " + decrypted);

s.close();
ss.close();
}

static String caesarEncrypt(String text, int key) {
String result = "";

for (char ch : text.toCharArray()) {
if (Character.isUpperCase(ch)) {
```

```
result += (char) ((ch - 'A' + key) % 26 + 'A');
} else if (Character.isLowerCase(ch)) {
result += (char) ((ch - 'a' + key) % 26 + 'a');
} else {
result += ch;
}
}

return result;
}

static String caesarDecrypt(String text, int key) {
return caesarEncrypt(text, 26 - key);
}

static String transpositionEncrypt(String text, int cols) {
int rows = (int) Math.ceil((double) text.length() / cols);
char[][] matrix = new char[rows][cols];

int k = 0;

for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
if (k < text.length())
matrix[i][j] = text.charAt(k++);
else
matrix[i][j] = 'X';
}
}

String cipher = "";

for (int j = 0; j < cols; j++) {
for (int i = 0; i < rows; i++) {
cipher += matrix[i][j];
}
}

return cipher;
}

static String transpositionDecrypt(String cipher, int cols) {
int rows = (int) Math.ceil((double) cipher.length() / cols);
char[][] matrix = new char[rows][cols];

int k = 0;

for (int j = 0; j < cols; j++) {
for (int i = 0; i < rows; i++) {
matrix[i][j] = cipher.charAt(k++);
}
}
```

```
String plain = "";

for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
plain += matrix[i][j];
}
}

return plain.replaceAll("X+$", "");
}
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class Client {
   public static void main(String[] args) throws Exception {

      Socket s = new Socket("localhost", 5000);

      BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
      PrintWriter out = new PrintWriter(s.getOutputStream(), true);

      Scanner sc = new Scanner(System.in);

      System.out.println("Choose Cipher:");
      System.out.println("1. Substitution Cipher Caesar");
      System.out.println("2. Transposition Cipher");
      System.out.print("Enter choice: ");
      String choice = sc.nextLine();

      System.out.print("Enter text: ");
      String text = sc.nextLine();

      System.out.print("Enter key: ");
      int key = sc.nextInt();

      out.println(choice);
      out.println(text);
      out.println(key);

      System.out.println("\n--- Result from Server ---");
      System.out.println(in.readLine());
      System.out.println(in.readLine());
      System.out.println(in.readLine());

      s.close();
   }
}
```